

Final Project Proposal

Topic:

traditional ML approach vs. GNN approach on ego-network of Facebook graph dataset

Dataset:

<https://snap.stanford.edu/data/ego-Facebook.html>

Abstract:

the most iconic real-world representation of graph structure is the social network. Whether it is on Facebook, Twitter or X, we individuals all represent a single node. And to ourselves, we are the central of a graph and all our friends represent other node with connection of edge. So, such ego-network is a perfect representation that we can apply our Machine Learning Models or Deep Learning Models to identify the social circles. But, only get the graph structure is not even close to identify the trend. Thus, the dataset on Stanford University provides a profile feature so that we can further make our models even more accurate. The whole idea is based on the research paper *Learning to Discover Social Circles in Ego Networks* which was written by Julian McAuley and Jure Leskovec from Stanford University. They successfully identify the social circle using the dataset. However, I don't want to just recreate their result or using the model they used. Instead, since I am on the path of learning machine learning and try to pivot myself from a civil engineer (which is major before I came to the US) to data scientist or even machine learning engineer, I really want to know what's the accuracy difference between using traditional Machine Learning Model and Graph Neural Networks. Thus, my project would be using the same dataset and tried different approach to compare the model metrics.

Keywords:

social analysis, ego-network, Machine Learning, Graph Neural Networks (GNN)

Brief Approach (traditional ML):

1. **Load the data:** from the website, we can get edges, feat, featnames, circles, and egofeat. Each represent different features to the social network. I would make a folder to contain these data and use library `os` to identify the relative path even though the whole folder is used by other people.
2. **the graph structure:** use the library `networkX` to put the data I downloaded into a graph structure. Then, use `pandas` to read the profile features.

3. **Convert the unstructured data into structured data for ML:** use of Node2Vec turns each node (profile) into a numerical vector (embedding). So that in the graph structure, I can get vector for each node.
4. **Combine embedding and profile features:** only profile embedding is not enough according to the conclusion of the research paper. Thus, we need to add profile features on top of it. Use `numpy` to stack both information.
5. **Prepare labels for comparison of both models:** from .circle file, we can then build labels by myself. I can then build a multiclass labels or multiple binary labels for Logistic Regression Model (Classification linear model).
6. **Train traditional ML model:** use `scikit-learn` model like Logistic Regression, Random Forest, or XGBoost, etc.
7. **Visualize the insight and result:** use t-SNE or PCA from `scikit-learn` to visualize the result.

Brief Approach (GNNs): (the first 2 steps are the same as ML approach)

1. **Load the data:** from the website, we can get edges, feat, featnames, circles, and ego feat. Each represent different features to the social network. I would make a folder to contain these data and use library `os` to identify the relative path even though the whole folder is used by other people.
2. **the graph structure:** use the library `networkX` to put the data I downloaded into a graph structure. Then, use `pandas` to read the profile features.
3. **Convert NetworkX graph into PyTorch Geometric:** GNN recognize PyTorch Geometric and make further process fairly easy.
4. **Define GNN model:** there are plenty type of GNN model and I can start defining Graph Convolutional Network (GCN)
5. **Train GNN model:** train the model using the data from PyTorch Geometric (similar like we trained the ML model)
6. **Visualization**

Compare and Conclude and Expectation

1. Node2Vec + ML (with profile feature) vs. Node2Vec + ML (without profile feature) vs. GNN
2. From the paper, I expect to get the conclusion that incorporating profile features significantly improves discovery accuracy.
3. Whether it is true or not that using DL model would be more accurate than ML model
4. The time I used for building the pipeline for ML and DL model